

Relazione Progetto

Intelligenza Artificiale

Corso di Laurea Magistrale

in

Ingegneria Informatica

A.A. 2023/2024

Mallamo Giuseppe (matr. 242767)

Tempo Fabrizio (matr. 247195)

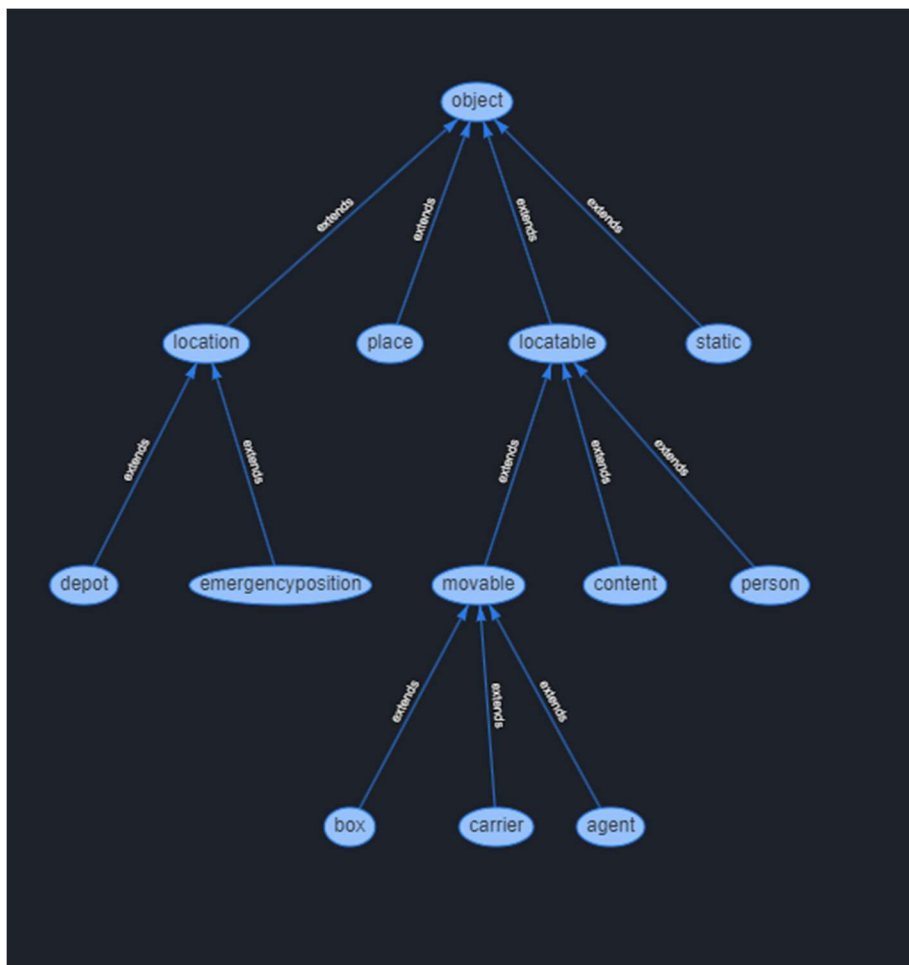
Indice

Indice	2
Modelling	3
Tipi	3
Predicati	5
Azioni	5
Classical Planning	9
Planner	9
Istance 1	12
Instance 2	14
Istance 3 (Bonus)	16
Temporal Planning & Robotics	19
Temporal planning	19
Robotics planning	25

Modelling

Tipi

Per modellare il problema dato usando il linguaggio PDDL è necessario definire un dominio che tenga conto delle specifiche richieste. A tale scopo, si è deciso di usare una gerarchia dei tipi in modo da distinguere i vari componenti del sistema che interagiscono tra loro e definire le loro proprietà, tale gerarchia è riportata di seguito.



Per comprendere il ruolo dei tipi usati se ne riporta una breve descrizione per ognuno:

1. Object: tipo padre degli altri tipi.
2. Location: rappresenta una zona del problema ovvero un luogo dove possono essere disposte persone, robot, carrelli e materiale.
3. Locatable: tipo che rappresenta tutti gli oggetti che possono trovarsi in una posizione determinata, ovvero una location.
4. Static: tipo che rappresenta gli elementi che non si spostano.

5. Place: tipo che rappresenta un posto di un carrello.
6. Depot: tipo che rappresenta il punto in cui sono tenuti tutti i beni necessari e i pacchi alla partenza.
7. Emergencyposition: tipo che rappresenta i luoghi in cui si trovano le persone bisognose di qualche cosa.
8. Movable: tipo che rappresenta gli oggetti che possono muoversi nella specifica del problema.
9. Box: rappresenta una scatola, la quale può contenere solo un contenuto e può essere alloggiata in un solo compartimento del carrello.
10. Carrier: Rappresenta un carrello, esso può essere agganciato ed essere spostato da un robot, possiede uno o più slot e può trovarsi in una determinata posizione.
11. Agent: rappresenta un robot, ovvero un'entità autonoma capace di muoversi, agganciare e spostare un carrello, riempire e svuotare una scatola per soddisfare il bisogno di una persona, inserire e rimuovere una scatola da un compartimento del carrello.
12. Content: rappresenta un contenuto, ovvero un oggetto che può essere inserito in una scatola e di cui le persone definite nel problema possono avere bisogno. Come assegnato nella traccia vi sono solo tre tipi di contenuto: cibo, strumenti e medicine, i quali saranno definiti nelle istanze dei problemi in seguito come oggetti di tipo contenuto.
13. Person: rappresenta una persona, la quale si trova in una determinata posizione e può necessitare di uno o più tipi di contenuto.

Questa gerarchia è stata definita nel file di dominio (domain.pddl) mediante l'uso della dichiarazione types come riportato nel seguente frammento di codice.

```
(define
  (domain emergency-domain)
  (:requirements :strips :typing :negative-preconditions)
  (:types
    place - object ; * Un posto di un carrello
    location - object ; * Un luogo in cui l'agente può spostarsi
    locatable - object ; * Un elemento che può essere collocato in una location
    static - object ; * Elementi che non si spostano
    movable - locatable ; * Un elemento che può essere spostato tra diverse location
    content - locatable ; * Il contenuto che può essere messo in una scatola
    person - locatable ; * Una persona che può richiedere delle cose
    box - movable ; * Una scatola riempibile con del content e che può essere trasportata
    carrier - movable ; * Il carrello su cui l'agente pone le scatole e le trasporta dove necessario
    agent - movable ; * L'agente che compie le azioni di trasporto dei pacchi
    depot - location ; * Il punto in cui sono tenuti tutti i beni necessari e i pacchi alla partenza
    emergencyposition - location ; * Luoghi in cui si trovano le persone bisognose di qualche cosa
  )
)
```

Predicati

Dopo aver definito i tipi per modellare lo stato del sistema è necessario definire i predicati, i quali saranno manipolati (affermati o negati) dalle azioni possibili al fine di raggiungere uno stato obiettivo (insieme di predicati da soddisfare). Dal momento che tali predicati devono specificare la configurazione del sistema dovranno rappresentare:

1. La locazione degli oggetti
2. Lo stato delle scatole: piene o vuote.
3. Il contenimento di un determinato contenuto o meno da una scatola.
4. I bisogni delle persone.
5. Lo stato dei compartimenti del carrier: alloggia una scatola o meno.
6. Lo stato di una scatola: è su un determinato carrello o meno.
7. L'appartenenza di un posto a uno specifico carrello.
8. La possibilità di caricare o meno una scatola su un carrello.

I predicati sono stati definiti nel file di dominio mediante la dichiarazione predicates della quale si riporta il codice di seguito:

```
(:predicates
  (at ?x-locatable ?ep-location) ; * Predicato che stabilisce se un elemento si trova in una location
  (empty ?b-box) ; * Predicato che stabilisce se una scatola è vuota
  (contains ?b-box ?c-content) ; * Predicato che stabilisce se una scatola contiene un determinato contenuto
  (need ?p-person ?c-content) ; * Predicato che stabilisce se una persona necessita di un contenuto
  (available ?p-place ?c-carrier) ; * Predicato che stabilisce se un posto del carrello è libero
  (on ?b-box ?c-carrier) ; * Predicato che stabilisce se una scatola è su un carrello
  (of ?p-place ?c-carrier) ; * Predicato che stabilisce se il posto è di un carrello
  (loadable ?b-box) ; * Predicato che stabilisce se una scatola può essere caricata su un carrello
)
```

Tutti questi predicati verranno usati in fase di definizione delle istanze dei problemi assegnati per dichiarare le condizioni iniziali del sistema.

Azioni

Le azioni possibili nel sistema, in accordo alla traccia, devono consentire che il robot possa:

1. Riempire una scatola con un determinato contenuto.
2. Consegnare il contenuto di una scatola a una persona che ne ha bisogno.
3. Caricare una scatola su un carrello.
4. Scaricare il contenuto di una scatola in un luogo.

5. Spostarsi con il carrello da una locazione ad un'altra in cui c'è una persona che ha bisogno di un contenuto presente in una delle scatole sul carrello.
6. Spostarsi con il carrello da una locazione ad un'altra.
7. Spostarsi da una locazione ad un'altra.

Ognuna di queste azioni è stata definita nel file di dominio mediante la dichiarazione action nella quale vengono specificate le precondizioni, ovvero i predicati che devono essere veri, e gli effetti, cioè i predicati da asserire o negare, in modo da portare il sistema in uno stato conforme all'azione applicata. Nelle successive sezioni si analizziamo gli effetti di ognuna di esse.

L'azione (1) avviene se la scatola b è vuota, è sul carrello cr e si trova al deposito d, il carrello cr e l'agente a si trovano anch'essi al deposito d e gli effetti sono che la scatola b non sarà più vuota ma conterrà il contenuto c.

```
(:action fill ; * Azione che effettua la riempitura di una scatola vuota con un contenuto
:parameters (?b - box ?d - depot ?c - content ?cr - carrier ?a - agent)
:precondition (and
    (empty ?b)
    (at ?cr ?d)
    (on ?b ?cr)
    (at ?b ?d)
    (at ?a ?d)
)
:effect (and
    (not (empty ?b))
    (contains ?b ?c)
)
)
```

L'azione (2) viene effettuata se la persona p, l'agente a e il carrello cr sono all'emergencyposition ep, inoltre la scatola b contiene il contenuto c ed è sul carrello cr e la persona p ha proprio bisogno del contenuto c. Gli effetti di questa azione sono: il fatto che la scatola b verrà svuotata del contenuto c e che la persona p non avrà più necessità di c.

```
(:action empty-box ; * Azione che effettua la consegna di un contenuto dietro una scatola ad una persona che ne ha bisogno
:parameters (?b - box ?c - content ?ep - emergencyposition ?p - person ?cr - carrier ?a - agent)
:precondition (and
    (at ?p ?ep)
    (at ?a ?ep)
    (at ?cr ?ep)
    (on ?b ?cr)
    (contains ?b ?c)
    (need ?p ?c)
)
:effect (and
    (empty ?b)
    (not (need ?p ?c))
    (not (contains ?b ?c))
)
)
```

L'azione (3) avviene se la scatola b, il carrello c e l'agente a sono alla location l, il posto p è per il carrello c ed è disponibile e se la scatola b non è sul carrello c ma è possibile caricarla. Gli effetti sono il fatto che dopo b sia su c e dunque non sia più disponibile, come il carrello c al posto p.

```
(:action load-box ; * Azione che effettua il carico di una scatola su un carrello
:parameters (?b - box ?c - carrier ?a - agent ?p - place ?l - location)
:precondition (and
  (at ?b ?l)
  (at ?c ?l)
  (at ?a ?l)
  (of ?p ?c)
  (available ?p ?c)
  (not (on ?b ?c))
  (loadable ?b)
)
:effect (and
  (on ?b ?c)
  (not (loadable ?b))
  (not (available ?p ?c))
)
)
```

L'azione (4) avviene se la scatola b è vuota, è sul carrello cr e dunque non può essere caricata su un altro carrello, se l'agente a e il carrello cr sono all'emergencyposition ep e se il posto p non è disponibile per cr. Gli effetti sono: b non si trova più su cr ma in ep; quindi, potrebbe essere caricata su un altro carrello, inoltre la postazione p diventa disponibile per cr.

```
(:action unload-box ; * Azione che effettua lo scarico di una scatola in un luogo
:parameters (?b - box ?cr - carrier ?a - agent ?p - place ?ep - emergencyposition)
:precondition (and
  (on ?b ?cr)
  (empty ?b)
  (not (loadable ?b))
  (at ?cr ?ep)
  (at ?a ?ep)
  (not (available ?p ?cr))
)
:effect (and
  (not (on ?b ?cr))
  (loadable ?b)
  (at ?b ?ep)
  (available ?p ?cr)
)
)
```

L'azione (5) si verifica se l'agente a, la scatola b e il carrello cr su cui si trova b si trovano alla location from mentre la persona p che si trova alla location to necessita del contenuto c che è dentro b. Come effetto dell'azione cr, a e b non saranno più in from ma in to.

```
(:action move-to-need ;
  :parameters (?p - person ?from - location ?to - location ?c - content ?b - box ?cr - carrier ?a - agent)
  :precondition (and
    (at ?a ?from)
    (at ?cr ?from)
    (at ?p ?to)
    (contains ?b ?c)
    (need ?p ?c)
    (at ?b ?from)
    (on ?b ?cr)
  )
  :effect (and
    (at ?cr ?to)
    (at ?a ?to)
    (at ?b ?to)
    (not(at ?b ?from))
    (not (at ?cr ?from))
    (not (at ?a ?from))
  )
)
```

L'azione (6) si verifica se l'agente a e il carrello c sono alla location from. Come effetto a e c saranno alla location to e non più a from.

```
(:action move ; * Azione che effettua lo spostamento dell'agente e del carrello da una location all'altra
  :parameters (?from - location ?to - location ?a - agent ?c - carrier)
  :precondition (and
    (at ?a ?from)
    (at ?c ?from)
  )
  :effect (and
    (at ?a ?to)
    (at ?c ?to)
    (not (at ?a ?from))
    (not (at ?c ?from))
  )
)
```

L'azione (7) avviene se l'agente a si trova nella location from. Dopo a non sarà più a from ma alla location to.

```
(:action move-agent ; * Azione che effettua lo spostamento dell'agente da una locazione all'altra
  :parameters (?from - location ?to - location ?a - agent)
  :precondition (at ?a ?from)
  :effect (and
    (at ?a ?to)
    (not (at ?a ?from))
  )
)
```

Classical Planning

Al fine di realizzare un planner che risolva le istanze dei problemi nelle sezioni successive si è utilizzata la libreria PDDL4J la quale consente di avere dei planner già pronti e da configurare o di costruirne uno personalizzato mediante l'estensione di alcune classi in linguaggio Java.

Planner

Il planner definito implementa l'algoritmo A* il quale combina insieme le strategie Greedy e UCS definendo un ordinamento per la ricerca pari alla somma della funzione euristica (h) e della funzione di costo (g).

L'implementazione di tale planner è stata resa possibile estendendo la classe AbstractPlanner della libreria PDDL4J con la classe ASP, la quale è presente nel file ASP.java. Per ragioni di brevità si riportano di seguito solo i metodi solve(Problem) e astar(Problem) i quali implementano l'algoritmo di ricerca e si rimanda ad una visione del file citato in precedenza, presente in allegato.

```
@Override
public Plan solve(final Problem problem) throws ProblemNotSupportedException {
    LOGGER.info("* Starting A* search \n");
    // Search a solution
    final long begin = System.currentTimeMillis();
    final Plan plan = this.astar(problem);
    final long end = System.currentTimeMillis();
    // If a plan is found update the statistics of the planner
    // and log search information
    if (plan != null) {
        LOGGER.info("* A* search succeeded\n");
        this.getStatistics().setTimeToSearch(end - begin);
    } else {
        LOGGER.info("* A* search failed\n");
    }
    // Return the plan found or null if the search fails.
    return plan;
}
```

```

/**
 * Search a solution plan for a planning problem using an A* search strategy.
 *
 * @param problem the problem to solve.
 * @return a plan solution for the problem or null if there is no solution
 * @throws ProblemNotSupportedException if the problem to solve is not supported by the planner.
 */
public Plan astar(Problem problem) throws ProblemNotSupportedException {
    // Check if the problem is supported by the planner
    if (!this.isSupported(problem)) {
        throw new ProblemNotSupportedException("Problem not supported");
    }

    // First we create an instance of the heuristic to use to guide the search
    final StateHeuristic heuristic = StateHeuristic.getInstance(this.getHeuristic(), problem);

    // We get the initial state from the planning problem
    final State init = new State(problem.getInitialState());

    // We initialize the closed list of nodes (store the nodes explored)
    final Set<Node> close = new HashSet<>();

    // We initialize the opened list to store the pending node according to function f
    final double weight = this.getHeuristicWeight();
    final PriorityQueue<Node> open = new PriorityQueue<>(100, new Comparator<Node>() {
        public int compare(Node n1, Node n2) {
            double f1 = weight * n1.getHeuristic() + n1.getCost();
            double f2 = weight * n2.getHeuristic() + n2.getCost();
            return Double.compare(f1, f2);
        }
    });

    // We create the root node of the tree search
    final Node root = new Node(init, null, -1, 0, heuristic.estimate(init, problem.getGoal()));

    // We add the root to the list of pending nodes
    open.add(root);
    Plan plan = null;

    // We set the timeout in ms allocated to the search
    final int timeout = this.getTimeout() * 1000;
    long time = 0;

```

```

// We start the search
while (!open.isEmpty() && plan == null && time < timeout) {

    // We pop the first node in the pending list open
    final Node current = open.poll();
    close.add(current);

    // If the goal is satisfied in the current node then extract the search and return it
    if (current.satisfy(problem.getGoal())) {
        return this.extractPlan(current, problem);
    } else { // Else we try to apply the actions of the problem to the current node
        for (int i = 0; i < problem.getActions().size(); i++) {
            // We get the actions of the problem
            Action a = problem.getActions().get(i);
            // If the action is applicable in the current node
            if (a.isApplicable(current)) {
                Node next = new Node(current);
                // We apply the effect of the action
                final List<ConditionalEffect> effects = a.getConditionalEffects();
                for (ConditionalEffect ce : effects) {
                    if (current.satisfy(ce.getCondition())) {
                        next.apply(ce.getEffect());
                    }
                }
                final double g = current.getCost() + h(a.getName());
                if (!close.contains(next)) {
                    next.setCost(g);
                    next.setParent(current);
                    next.setAction(i);
                    next.setHeuristic(heuristic.estimate(next, problem.getGoal()));
                    open.add(next);
                }
            }
        }
    }
}

// Finally, we return the search computed or null if no search was found
return plan;
}

```

Inoltre, si è reso necessario definire anche la classe `Node`, la quale estende la classe `State` di `PDDL4J` e rappresenta un nodo dell'albero di ricerca generato. Tale classe contiene le informazioni necessarie alla ricerca come il costo del nodo raggiunto, la profondità, l'azione che l'ha generato, etc. Può essere visualizzata nel file `Node.java`.

L'euristica usata per risolvere le istanze dei problemi descritti nella traccia, presenti nelle sezioni successive, è realizzata dal metodo `h(String action)` presente nel file `ASP.java` ed è abbastanza semplice; attribuisce un peso leggermente maggiore alle azioni *move* e *unload-box*.

```

/**
 * Metodo che assegna vari costi all'azione che sta per essere eseguita
 *
 * @param action il nome dell'azione eseguita
 * @return costo dell'azione
 */
private int h(String action){
    if(action.equals("move")){
        return 3;
    } else if (action.equals("unload-box")) {
        return 4;
    }
    return 1;
}

```

Inoltre, si può notare come il valore dell'utility sia dato dalla somma tra la profondità raggiunta e il valore dell'euristica.

Instance 1

Le condizioni iniziali della prima istanza riportata nella traccia sono:

1. Cinque scatole posizionate nella zona depot.
2. Cibo, attrezzi e medicine in numero sufficiente posizionati al depot.
3. Le persone p1 e p2 nella stessa zona, diciamo l1, e p3 in una zona diversa, diciamo l2. p1 richiede di cibo e medicine, p2 richiede solo medicine e p3 solo cibo.
4. Un singolo robot e un singolo carrello al depot
5. Il carrello ha capacità di carico di massimo quattro scatole.

Per modellare tale problema si è definito un file di problema in linguaggio PDDL nel quale mediante la dichiarazione `objects` si istanziano gli oggetti necessari al problema in accordo con la gerarchia definita. Successivamente mediante la dichiarazione `init` e `goal` si specificano le condizioni iniziali, predicati veri all'inizio, e gli obiettivi, predicati che devono essere veri alla fine. Di seguito è riportato il codice del file `problem1.pddl` il quale è presente in allegato.

```

(define
  (problem problem1)
    (:domain emergency-domain)
    (:objects
      b1 b2 b3 b4 b5 - box
      place1 place2 place3 place4 - place
      depot - depot
      l1 l2 - emergencyposition
      p1 p2 p3 - person
      a - agent
      c - carrier
      medicine food - content
    )

  (:init
    (empty b1)
    (empty b2)
    (empty b3)
    (empty b4)
    (empty b5)
    (loadable b1)
    (loadable b2)
    (loadable b3)
    (loadable b4)
    (loadable b5)
    (of place1 c)
    (of place2 c)
    (of place3 c)
    (of place4 c)
    (available place1 c)
    (available place2 c)
    (available place3 c)
    (available place4 c)
    (at c depot)
    (at a depot)
    (at b1 depot)
    (at b2 depot)
    (at b3 depot)
    (at b4 depot)
    (at b5 depot)
    (at p1 l1)
    (at p2 l1)
    (at p3 l2)
    (need p1 food)
    (need p1 medicine)
    (need p2 medicine)
    (need p3 food)
  )
  (:goal (and (not (need p1 food)) (not (need p1 medicine))(not (need p2 medicine)) (not (need p3 food)))))
)

```

La risoluzione di questa prima istanza con il planner e l'euristica definiti in precedenza avviene in tempi brevissimi, 038 s, e con un uso di memoria centrale decisamente minimo di soli 1,01 MBytes. L'output fornito è il seguente:

```

problem instantiation done successfully (208 actions, 61 fluents)

* Starting A* search
* A* search succeeded

found plan as follows:

00: (      load-box b1 c a place1 depot) [0]
01: (      load-box b4 c a place2 depot) [0]
02: (      fill b1 depot medicine c a) [0]
03: (      fill b4 depot food c a) [0]
04: (      load-box b5 c a place3 depot) [0]
05: (      fill b5 depot medicine c a) [0]
06: (move-to-need p1 depot l1 medicine b1 c a) [0]
07: (      empty-box b1 medicine l1 p1 c a) [0]
08: (      empty-box b5 medicine l1 p2 c a) [0]
09: (      empty-box b4 food l1 p1 c a) [0]
10: (      move l1 depot a c) [0]
11: (      fill b4 depot food c a) [0]
12: (      move-to-need p3 depot l2 food b4 c a) [0]
13: (      empty-box b4 food l2 p3 c a) [0]

time spent:      0,03 seconds parsing
                  0,15 seconds encoding
                  0,20 seconds searching
                  0,38 seconds total time

memory used:      1,01 MBytes for problem representation
                  0,00 MBytes for searching
                  1,01 MBytes total

```

Instance 2

Le condizioni iniziali della seconda istanza riportata nella traccia sono:

1. Due robot e due carrelli posizionati al depot.
2. Tre scatole posizionate al depot.
3. Capacità del carrello pari a due.
4. Sei persone: p1 richiede cibo o attrezzi, p2 richiede medicine, p3 richiede medicine, p4 richiede medicine e cibo, p5 e p6 richiedono tutto.
5. Le prime due persone sono nella stessa zona mentre le altre in posizione diversa.

Per modellare il problema si è definito un file di problema `problem2.ppdl` del tutto analogo al precedente ma con specifiche diverse. Tuttavia, si sottolinea che la realizzazione della condizione esclusiva sulle richieste della persona p1 non è stato realizzabile dal momento che il parser intrinseco nella libreria PDDL4J non permette di inserire una condizione in `or` nei goal. Per tale motivo le condizioni sono state messe in `and` al resto dei goal. Di seguito si riporta tale file presente sempre in allegato al quale rimandiamo la visione.

```

(define
  (problem instance2)
    (:domain emergency-domain)
    (:objects
      a1 a2 - agent
      c1 c2 - carrier
      b1 b2 b3 - box
      place1 place2 place3 place4 - place
      p1 p2 p3 p4 p5 p6 - person
      food tool medicine - content
      depot - depot
      l1 l2 l3 l4 l5 - emergencyposition
    )

```

```

  (:init
    (empty b1)
    (empty b2)
    (empty b3)
    (loadable b1)
    (loadable b2)
    (loadable b3)
    (of place1 c1)
    (of place2 c1)
    (of place3 c2)
    (of place4 c2)
    (available place1 c1)
    (available place2 c1)
    (available place3 c2)
    (available place4 c2)
    (at a1 depot)
    (at a2 depot)
    (at c1 depot)
    (at c2 depot)
    (at b1 depot)
    (at b2 depot)
    (at b3 depot)
    (at p1 l1)
    (at p2 l1)
    (at p3 l2)
    (at p4 l3)
    (at p5 l4)
    (at p6 l5)
    (need p1 food)
    (need p1 tool)
    (need p2 medicine)
    (need p3 medicine)
    (need p4 medicine)
    (need p4 food)
    (need p5 food)
    (need p5 tool)
    (need p5 medicine)
    (need p6 food)
    (need p6 tool)
    (need p6 medicine)
  )

```

```

)
(:goal (and (not (need p1 food))
            (not (need p1 tool))
            (not (need p2 medicine))
            (not (need p3 medicine))
            (not (need p4 medicine))
            (not (need p4 food))
            (not (need p5 food))
            (not (need p5 tool))
            (not (need p5 medicine))
            (not (need p6 food))
            (not (need p6 tool))
            (not (need p6 medicine))
            )
)
)

```

A differenza del caso precedente la risoluzione di questa seconda istanza richiede qualche risorsa in più; infatti, termina in circa 88 secondi e occupa 7,24 MBytes. Inoltre, il risultato del planner è composto da 54 passi.

Istance 3 (Bonus)

Le condizioni iniziali della terza istanza riportata nella traccia sono:

1. Quattro scatole al depot.
2. Otto persone: p1 richiede cibo o attrezzi, p2 richiede medicine, p3 richiede medicine, p4 richiede medicine e cibo, p5, p6, p7 e p8 richiedono tutto.
3. Le prime due persone sono posizionate nella stessa zona le restanti in zone diverse.
4. Due robot.
5. Due carrelli con capienza due.

Per modellare il problema si è definito un file di problema `problem3.pddl` del tutto analogo al precedente ma con specifiche diverse reperibile sempre in allegato. Come nel caso precedente anche in questa terza istanza è stato impossibile inserire una condizione in `or` nei goal per le motivazioni discusse al punto precedente e le tali condizioni sono state poste in `and` con le restanti.

```

(define
  (problem instance2)
    (:domain emergency-domain)
    (:objects
      a1 a2 - agent
      c1 c2 - carrier
      b1 b2 b3 b4 - box
      place1 place2 place3 place4 - place
      p1 p2 p3 p4 p5 p6 p7 p8 - person
      food tool medicine - content
      depot - depot
      l1 l2 l3 l4 l5 l6 l7 - emergencyposition
    )

```

```

  (:init
    (empty b1)
    (empty b2)
    (empty b3)
    (empty b4)
    (loadable b1)
    (loadable b2)
    (loadable b3)
    (loadable b4)
    (of place1 c1)
    (of place2 c1)
    (of place3 c2)
    (of place4 c2)
    (available place1 c1)
    (available place2 c1)
    (available place3 c2)
    (available place4 c2)
    (at a1 depot)
    (at a2 depot)
    (at c1 depot)
    (at c2 depot)
    (at b1 depot)
    (at b2 depot)
    (at b3 depot)
    (at b4 depot)
    (at p1 l1)
    (at p2 l1)
    (at p3 l2)
    (at p4 l3)
    (at p5 l4)
    (at p6 l5)
    (at p7 l6)
    (at p8 l7)
    (need p1 food)
    (need p1 tool)
    (need p2 medicine)
    (need p3 medicine)
    (need p4 medicine)
  )

```

```

        (need p4 food)
        (need p5 food)
        (need p5 tool)
        (need p5 medicine)
        (need p6 food)
        (need p6 tool)
        (need p6 medicine)
        (need p7 food)
        (need p7 tool)
        (need p7 medicine)
        (need p8 food)
        (need p8 tool)
        (need p8 medicine)
    )
    (:goal (and      (not (need p1 food))
                     (not (need p1 tool))
                     (not (need p2 medicine))
                     (not (need p3 medicine))
                     (not (need p4 medicine))
                     (not (need p4 food))
                     (not (need p5 food))
                     (not (need p5 tool))
                     (not (need p5 medicine))
                     (not (need p6 food))
                     (not (need p6 tool))
                     (not (need p6 medicine))
                     (not (need p7 food))
                     (not (need p7 tool))
                     (not (need p7 medicine))
                     (not (need p8 food))
                     (not (need p8 tool))
                     (not (need p8 medicine))
                )
    )
)

```

La risoluzione di questa terza istanza richiede un numero sostanzialmente più elevato di risorse di calcolo, impiegando poco più di 9 minuti e usando 16,71 MByte di memoria centrale, per trovare però un output di soli 77 passi.

Temporal Planning & Robotics

Temporal planning

Per convertire il dominio definito nei punti precedenti in maniera tale da poter effettuare temporal planning, in cui sono presenti delle azioni come *durative-actions*, è necessario effettuare alcune modifiche strutturali al dominio. Per prima cosa si è definito un insieme di *functions* per tenere traccia di alcune quantità.

```
(:functions
  (path-cost)
  (weight ?c - content)
  (box-weight ?b - box)
  (carrier-weight ?c - carrier )
)
```

In particolare:

- **path-cost**: mantiene il costo del piano fino allo stato corrente;
- **weight**: indica il peso di un bene;
- **box-weight**: tiene traccia del peso complessivo di una scatola;
- **carrier-weight**: tiene traccia del peso complessivo di un carrello.

Per evitare che l'agente potesse essere occupato in più attività contemporaneamente si è definito un predicato **free-agent** che permette di stabilire se l'agente è impegnato in un'altra attività.

In fine, si sono convertite tutte le azioni del dominio in **durative-actions** e in questo senso è stato necessario definire in maniera opportuna la durata di ogni azione.

- L'azione **fill** ha durata pari al peso del contenuto che si deve inserire nella scatola;
- L'azione **empty-box** ha durata pari al peso del contenuto della scatola;
- L'azione **load-box** ha durata pari al peso della scatola che si deve caricare;
- L'azione **unload-box** ha durata pari al peso della scatola che si deve scaricare;
- L'azione **move-to-need** ha durata pari al peso complessivo del carrello che si sposta nel movimento;

- L'azione **move** ha durata pari al peso complessivo del carrello che si sposta nel movimento;
- L'azione **move-agent** ha durata costante pari a 3.

Di seguito sono riportate tutte le **durative-actions** definite nel nuovo dominio

```
(:durative-action fill
:parameters (?b - box ?d - depot ?c - content ?cr - carrier ?a - agent)
:duration (= ?duration (weight ?c))
:condition (and
  (at start (free-agent ?a))
  (at start (empty ?b))
  (over all (loadable ?b))
  (over all (at ?cr ?d))
  (over all (at ?b ?d))
  (over all (at ?a ?d))
)
:effect (and
  (at start (not (free-agent ?a)))
  (at end (increase (box-weight ?b) (weight ?c)))
  (at end (increase path-cost (* (box-weight ?b) 2)))
  (at end (contains ?b ?c))
  (at end (not (empty ?b)))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action empty-box
:parameters (?b - box ?c - content ?ep - emergencyposition ?p - person ?cr - carrier ?a - agent)
:duration (= ?duration (weight ?c))
:condition (and
  (at start (free-agent ?a))
  (at start (contains ?b ?c))
  (over all (at ?p ?ep))
  (over all (at ?a ?ep))
  (over all (at ?cr ?ep))
  (over all (on ?b ?cr))
  (over all (need ?p ?c))
)
:effect (and
  (at start (not (free-agent ?a)))
  (at start (not (contains ?b ?c)))
  (at end (decrease (box-weight ?b) (weight ?c)))
  (at end (decrease (carrier-weight ?cr) (weight ?c)))
  (at end (empty ?b))
  (at end (not (need ?p ?c)))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action load-box
:parameters (?b - box ?c - carrier ?a - agent ?p - place ?l - location)
:duration (= ?duration (box-weight ?b))
:condition (and
  (at start (free-agent ?a))
  (over all (at ?b ?l))
  (over all (at ?c ?l))
  (over all (at ?a ?l))
  (over all (of ?p ?c))
  (at start (available ?p ?c))
  (at start (not (on ?b ?c)))
  (at start (loadable ?b))
)
:effect (and
  (at start (not (free-agent ?a)))
  (at end (increase (carrier-weight ?c) (box-weight ?b)))
  (at end (increase path-cost (+ (box-weight ?b) 3)))
  (at end (on ?b ?c))
  (at end (not (loadable ?b)))
  (at end (not (available ?p ?c)))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action unload-box
:parameters (?b - box ?cr - carrier ?a - agent ?p - place ?ep - emergencyposition)
:duration (= ?duration (box-weight ?b) )
:condition (and
  (at start (free-agent ?a))
  (at start (on ?b ?cr))
  (at start (empty ?b))
  (at start (not (loadable ?b)))
  (over all (at ?cr ?ep))
  (over all (at ?a ?ep))
  (at start (not (available ?p ?cr)))
)
:effect (and
  (at start (not(free-agent ?a)))
  (at end (increase path-cost 8))
  (at end (not(on ?b ?cr)))
  (at end (loadable ?b))
  (at end (at ?b ?ep))
  (at end (available ?p ?cr))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action move-to-need
:parameters (?p - person ?from - location ?to - location ?c - content ?b - box ?cr - carrier ?a - agent)
:duration (= ?duration (carrier-weight ?cr) )
:condition (and
  (at start (free-agent ?a))
  (at start (at ?cr ?from))
  (at start (at ?a ?from))
  (over all (on ?b ?cr))
  (over all (at ?p ?to))
  (over all (contains ?b ?c))
  (over all (need ?p ?c))
)
:effect (and
  (at start (not(free-agent ?a)))
  (at start (not (at ?cr ?from)))
  (at start (not (at ?a ?from)))
  (at end (increase path-cost (* (carrier-weight ?cr) 5)))
  (at end (at ?a ?to))
  (at end (at ?cr ?to))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action move
:parameters (?from - location ?to - location ?a - agent ?cr - carrier)
:duration (= ?duration (carrier-weight ?cr) )
:condition (and
  (at start (free-agent ?a))
  (at start (at ?a ?from))
  (at start (at ?cr ?from))
)
:effect (and
  (at start (not(free-agent ?a)))
  (at start (not (at ?a ?from)))
  (at start (not (at ?cr ?from)))
  (at start (increase path-cost (* (carrier-weight ?cr) 10)))
  (at end (at ?a ?to))
  (at end (at ?cr ?to))
  (at end (free-agent ?a))
)
)
```

```
(:durative-action move-agent
  :parameters (?from - location ?to - location ?a - agent)
  :duration (= ?duration 3)
  :condition (and
    (at start (free-agent ?a))
    (at start (at ?a ?from))
  )
  :effect (and
    (at start (not(free-agent ?a)))
    (at start (not (at ?a ?from)))
    (at start (increase path-cost 15))
    (at end (at ?a ?to))
    (at end (free-agent ?a))
  )
)
```

Di seguito riportiamo anche la prima istanza per mostrare le modifiche apportate su di essa per effettuare temporal planning.

```
(define
  (problem instance1)
  (:domain emergency-handling)
  (:objects
    b1 b2 b3 b4 b5 - box
    place1 place2 place3 place4 - place
    depot - depot
    l1 l2 - emergencyposition
    p1 p2 p3 - person
    a - agent
    c - carrier
    medicine food - content
  )
)
```

```
(:init
  (empty b1)
  (empty b2)
  (empty b3)
  (empty b4)
  (empty b5)
  (loadable b1)
  (loadable b2)
  (loadable b3)
  (loadable b4)
  (loadable b5)
  (of place1 c)
  (of place2 c)
  (of place3 c)
  (of place4 c)
  (available place1 c)
  (available place2 c)
  (available place3 c)
  (available place4 c)
  (at c depot)
  (at a depot)
  (at b1 depot)
  (at b2 depot)
  (at b3 depot)
  (at b4 depot)
  (at b5 depot)
  (at p1 l1)
  (at p2 l1)
  (at p3 l2)
  (need p1 food)
  (need p1 medicine)
  (need p2 medicine)
  (need p3 food)
  (= (weight food) 5)
  (= (weight medicine) 3)
  (= (box-weight b1) 0)
  (= (box-weight b2) 0)
  (= (box-weight b3) 0)
  (= (box-weight b4) 0)
  (= (box-weight b5) 0)
  (= (carrier-weight c) 0)
  (= (path-cost) 0)
)
```

```
(:goal (and (not (need p1 food)) (not (need p1 medicine))(not (need p2 medicine)) (not (need p3 food))))

(:metric minimize (path-cost)
)
```

Utilizzando il solver per temporal planning lpg-td della libreria planutils otteniamo il piano seguente:

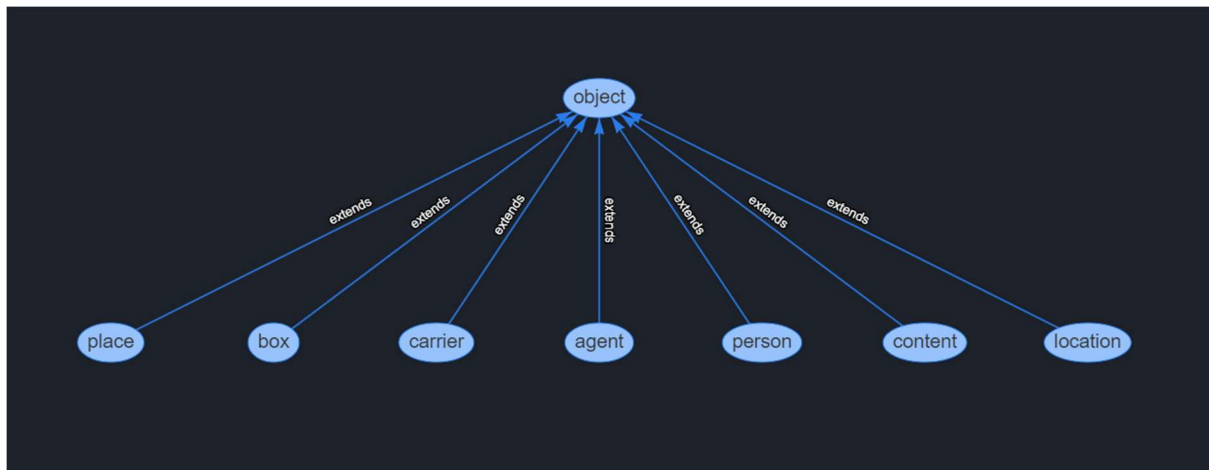
```
Plan computed:
  Time: (ACTION) [action Duration; action Cost]
0.0003: (FILL B4 DEPOT MEDICINE C A) [D:1.0000; C:0.1000]
1.0005: (LOAD-BOX B4 C A PLACE4 DEPOT) [D:1.0000; C:4.0000]
2.0008: (FILL B1 DEPOT MEDICINE C A) [D:1.0000; C:0.1000]
3.0010: (FILL B3 DEPOT FOOD C A) [D:2.0000; C:0.1000]
5.0012: (LOAD-BOX B1 C A PLACE2 DEPOT) [D:1.0000; C:4.0000]
6.0015: (MOVE-TO-NEED P2 DEPOT L1 MEDICINE B4 C A) [D:2.0000; C:10.0000]
8.0017: (EMPTY-BOX B4 MEDICINE L1 P2 C A) [D:1.0000; C:0.1000]
9.0020: (EMPTY-BOX B1 MEDICINE L1 P1 C A) [D:1.0000; C:0.1000]
10.0028: (FILL B1 DEPOT FOOD C A) [D:2.0000; C:0.1000]
12.0030: (LOAD-BOX B1 C A PLACE4 DEPOT) [D:2.0000; C:5.0000]
14.0033: (FILL B2 DEPOT FOOD C A) [D:2.0000; C:0.1000]
16.0035: (LOAD-BOX B2 C A PLACE1 DEPOT) [D:2.0000; C:5.0000]
18.0037: (MOVE DEPOT L1 A C) [D:4.0000; C:40.0000]
22.0040: (EMPTY-BOX B1 FOOD L1 P1 C A) [D:2.0000; C:0.1000]
24.0042: (MOVE L1 L2 A C) [D:2.0000; C:20.0000]
26.0045: (EMPTY-BOX B2 FOOD L2 P3 C A) [D:2.0000; C:0.1000]

Solution found:
Total time:      1.01
Search time:     0.04
Actions:         18
Execution cost:  96.00
Duration:        28.000
Plan quality:    96.000
  Plan file:      plan_problem1.pddl_1.SOL
```

Da cui possiamo notare un numero di azioni totali eseguite ridotto e un tempo totale di esecuzione basso.

Robotics planning

Per integrare il problema in un framework di robotics planning come plansys2 è necessario per prima cosa apportare una serie di modifiche al dominio per renderlo compatibile. Per prima cosa è necessario eliminare la gerarchia di tipi (non supportata da plansys2) andando a mantenere quelli strettamente necessari ottenendo i tipi in figura.



Successivamente, avendo eliminato la gerarchia dei tipi è stato necessario definire per tutti i predicati che facevano riferimenti a tipi più astratti nuovi predicati per ogni sottotipo concreto di quel tipo astratto. Abbiamo ottenuto così i predicati:

```
(:predicates
  (at_box ?b - box ?l - location)
  (at_carrier ?c - carrier ?l - location)
  (at_agent ?a - agent ?l - location)
  (at_content ?c - content ?l - location)
  (at_person ?p - person ?l - location)
  (empty ?b - box) 2 1 1
  (contains ?b - box ?c - content) 2 1 1
  (need ?p - person ?c - content) 2 1
  (available ?p - place ?c - carrier) 2 1 1
  (on ?b - box ?c - carrier) 4 1 1
  (of ?p - place ?c - carrier) 1
  (loadable ?b - box) 3 1 1
  (free-agent ?a - agent) 7 7 7
)
```

Essendo che plansys2 non supporta le functions è stato necessario quindi rimuoverle e di conseguenza andare a modificare le azioni. Questa rimozione ha impattato principalmente la definizione delle durate delle singole azioni. Le durate delle azioni sono state fissate a:

- **3** per lo spostamento dell'agente a prescindere dal tipo di movimento;
- **2** per le operazioni di carico e scarico delle scatole;

- 1 per le operazioni di svuotamento e riempimento delle scatole.

Per integrare il problema in plansys2 è inoltre stato necessario realizzare un pacchetto compatibile in cui:

1. Le azioni sono state definite come delle *fake-action* (file C++ che definisce una azione per plansys2);
2. Implementare un file python contenente la descrizione di avvio del pacchetto e il mapping alle fake-actions;
3. Definire le dipendenze e le operazioni di costruzione del pacchetto in un file di tipo CMake.

Le fake-actions sono state realizzate sulla base del tutorial “simple example” fornito dal team di plansys2. Di seguito la fake-action associata all’azione move.

```
#include <memory>
#include <algorithm>

#include "plansys2_executor/ActionExecutorClient.hpp"

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

using namespace std::chrono_literals;

class MoveAction : public plansys2::ActionExecutorClient
{
public:
    MoveAction()
    : plansys2::ActionExecutorClient("move", 250ms)
    {
        progress_ = 0.0;
    }

private:
    void do_work()
    {
        if (progress_ < 1.0) {
            progress_ += 0.02;
            send_feedback(progress_, "Move running");
        } else {
            finish(true, 1.0, "Move completed");
        }

        progress_ = 0.0;
        std::cout << std::endl;
    }

    std::cout << "\r\e[K" << std::flush;
    std::cout << "Moving ... [" << std::min(100.0, progress_ * 100.0) << "%] " <<
        std::flush;
};

float progress_;

int main(int argc, char ** argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<MoveAction>();

    node->set_parameter(rclcpp::Parameter("action_name", "move"));
    node->trigger_transition(lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE);

    rclcpp::spin(node->get_node_base_interface());

    rclcpp::shutdown();

    return 0;
}
```

Verrà riportato ora il codice python che implementa la descrizione dell'avvio del pacchetto e il mapping delle fake-actions.

```
import os

from ament_index_python.packages import get_package_share_directory

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
```

```
def generate_launch_description():
    # Get the launch directory
    example_dir = get_package_share_directory('emergency-handling-plansys2')
    namespace = LaunchConfiguration('namespace')

    declare_namespace_cmd = DeclareLaunchArgument(
        'namespace',
        default_value='',
        description='Namespace')

    plansys2_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(
            get_package_share_directory('plansys2_bringup'),
            'launch',
            'plansys2_bringup_launch_monolithic.py')),
        launch_arguments={
            'model_file': example_dir + '/pddl/domain.pddl',
            'namespace': namespace
        }.items())

    # Specify the actions
    move_cmd = Node(
        package='emergency-handling-plansys2',
        executable='move_node',
        name='move_node',
        namespace=namespace,
        output='screen',
        parameters=[])

    move_to_need_cmd = Node(
        package='emergency-handling-plansys2',
        executable='move_to_need_node',
        name='move_to_need_node',
        namespace=namespace,
        output='screen',
        parameters=[])
```

```

move_agent_cmd = Node(
    package='emergency-handling-plansys2',
    executable='move_agent_node',
    name='move_agent_node',
    namespace=namespace,
    output='screen',
    parameters=[])

load_box_cmd = Node(
    package='emergency-handling-plansys2',
    executable='load_box_node',
    name='load_box_node',
    namespace=namespace,
    output='screen',
    parameters=[])

unload_box_cmd = Node(
    package='emergency-handling-plansys2',
    executable='unload_box_node',
    name='unload_box_node',
    namespace=namespace,
    output='screen',
    parameters=[])

empty_box_cmd = Node(
    package='emergency-handling-plansys2',
    executable='empty_box_node',
    name='empty_box_node',
    namespace=namespace,
    output='screen',
    parameters=[])

fill_cmd = Node(
    package='emergency-handling-plansys2',
    executable='fill_node',
    name='fill_node',
    namespace=namespace,
    output='screen',
    parameters=[])

```

```

# Create the launch description and populate
ld = LaunchDescription()

ld.add_action(declare_namespace_cmd)

# Declare the launch options
ld.add_action(plansys2_cmd)

ld.add_action(move_cmd)
ld.add_action(move_to_need_cmd)
ld.add_action(move_agent_cmd)
ld.add_action(fill_cmd)
ld.add_action(empty_box_cmd)
ld.add_action(load_box_cmd)
ld.add_action(unload_box_cmd)

return ld

```

E infine il file CMake per la compilazione del pacchetto.

```
cmake_minimum_required(VERSION 3.5)
project(emergency-handling-plansys2)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(plansys2_msgs REQUIRED)
find_package(plansys2_executor REQUIRED)

set(CMAKE_CXX_STANDARD 17)

set(dependencies
  rclcpp
  rclcpp_action
  plansys2_msgs
  plansys2_executor
)

add_executable(empty_box_node src/empty_box.cpp)
ament_target_dependencies(empty_box_node ${dependencies})

add_executable(fill_node src/fill.cpp)
ament_target_dependencies(fill_node ${dependencies})

add_executable(load_box_node src/load_box.cpp)
ament_target_dependencies(load_box_node ${dependencies})

add_executable(move_node src/move.cpp)
ament_target_dependencies(move_node ${dependencies})

add_executable(move_to_need_node src/move_to_need.cpp)
ament_target_dependencies(move_to_need_node ${dependencies})

add_executable(move_agent_node src/move_agent.cpp)
ament_target_dependencies(move_agent_node ${dependencies})

add_executable(unload_box_node src/unload_box.cpp)
ament_target_dependencies(unload_box_node ${dependencies})

install(DIRECTORY launch pddl DESTINATION share/${PROJECT_NAME})
```

```
install(TARGETS
  empty_box_node
  fill_node
  load_box_node
  move_node
  move_to_need_node
  move_agent_node
  unload_box_node
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION lib/${PROJECT_NAME}
)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  ament_lint_auto_find_test_dependencies()

  find_package(ament_cmake_gtest REQUIRED)
endif()

ament_export_dependencies(${dependencies})

ament_package()
```

Ultimo passo prima di eseguire plansys2 sul nostro problema per ottenere il robotic plan è quello di trasformare l'istanza del problema in un file di comandi che è compatibile con il terminare Plansys2.

Di seguito è riportato il file commands per l'istanza 1.

```
set instance b1 box
set instance b2 box
set instance b3 box
set instance b4 box
set instance b5 box
set instance place1 place
set instance place2 place
set instance place3 place
set instance place4 place
set instance depot location
set instance l1 location
set instance l2 location
set instance p1 person
set instance p2 person
set instance p3 person
set instance a agent
set instance c carrier
set instance medicine content
set instance food content
set predicate (free-agent a)
set predicate (empty b1)
set predicate (empty b2)
set predicate (empty b3)
set predicate (empty b4)
set predicate (empty b5)
set predicate (loadable b1)
set predicate (loadable b2)
set predicate (loadable b3)
set predicate (loadable b4)
set predicate (loadable b5)
set predicate (of place1 c)
set predicate (of place2 c)
set predicate (of place3 c)
set predicate (of place4 c)
set predicate (available place1 c)
set predicate (available place2 c)
set predicate (available place3 c)
set predicate (available place4 c)
set predicate (at_carrier c depot)
set predicate (at_agent a depot)
set predicate (at_person p1 l1)
set predicate (at_person p2 l1)
set predicate (at_person p3 l2)
set predicate (at_box b1 depot)
set predicate (at_box b2 depot)
set predicate (at_box b3 depot)
set predicate (at_box b4 depot)
set predicate (at_box b5 depot)
set predicate (need p1 food)
set predicate (need p1 medicine)
set predicate (need p2 medicine)
set predicate (need p3 food)
set goal (and (not (need p1 food))(not (need p1 medicine))(not (need p2 medicine))(not (need p3 food)))
```

Fatto ciò, compilato il package, è tutto pronto per generare il robotic plan tramite plansys2. Il risultato del solver è il seguente.

```
[INFO] [1700905510.848558525] [terminal]: No problem file specified.  
ROS2 Planning System console. Type "quit" to finish  
> source ./launch/commands  
> run plan-file plan.plan  
The plan read from "plan.plan" is  
0:      (fill b2 depot food c a)          [2]  
2:      (load-box b2 c a place3 depot)    [2]  
4:      (fill b5 depot medicine c a)      [1]  
5:      (load-box b5 c a place1 depot)    [1]  
6:      (fill b1 depot food c a)          [2]  
8:      (fill b4 depot food c a)          [2]  
10:     (load-box b1 c a place2 depot)    [2]  
12:     (fill b3 depot medicine c a)      [1]  
13:     (load-box b3 c a place4 depot)    [1]  
14:     (move-to-need p1 depot l1 food b2 c a) [6]  
20:     (empty-box b2 food l1 p1 c a)      [2]  
22:     (empty-box b5 medicine l1 p1 c a)  [1]  
23:     (empty-box b3 medicine l1 p2 c a)  [1]  
24:     (move-to-need p3 l1 l2 food b1 c a) [2]  
26:     (empty-box b1 food l2 p3 c a)      [2]  
[INFO] [1700905736.762803858] [executor_client]: Plan Succeeded  
  
Successful finished  
>
```

Come possiamo notare il piano termina con successo per cui possiamo dire che il dominio e l'istanza del problema sono state definite in modo opportuno.